



UNIVERSITÉ ABDELMALEK ESSAÂDI
Faculté des Sciences et Techniques de Tanger — Master IASD

Adaptation de l'Algorithme de Lichtenberg (LA) au Problème du Voyageur de Commerce (TSP)

*Encodage par Clés Aléatoires, Hybridation Mémétique 2-opt
et Recherche Locale Itérée (Double-Bridge)*

Module : Métaheuristiques
Réalisé dans le cadre d'un projet académique de Master
Année universitaire 2025 / 2026

Table des matières

Table des matières	2
1. Introduction et contexte	3
2. Rappel théorique	3
2.1 L'Algorithme de Lichtenberg (LA)	3
2.2 Le Problème du Voyageur de Commerce (TSP)	3
2.3 Encodage par Clés Aléatoires (Random Keys)	4
3. Le travail original des auteurs : portée et limite	5
3.1 L'inspiration physique	5
3.2 L'architecture originale	5
3.3 La limite : une validation restreinte aux fonctions mathématiques continues	5
4. Architecture de la solution	7
4.1 Fidélité à l'implémentation MATLAB originale	7
4.2 Structure modulaire du code	7
5. Diagnostic d'un problème structurel : la stagnation prématurée	8
6. Hybridation LA-ILS proposée	8
6.1 Recherche locale 2-opt (étape mémétique)	8
6.2 Perturbation double-bridge (échappement de stagnation)	8
7. Étude d'ablation : la preuve expérimentale	9
7.1 Résultats de l'ablation	9
7.2 Interprétation	9
8. Résultats de l'hybride complet sur berlin52	11
9. Résultats multi-graines de l'hybride complet (5 instances)	11
10. Limites et discussion	12
10.1 Sensibilité à la plateforme	13
10.2 Nature réelle de l'hybride	13
10.3 Pistes d'amélioration	13
11. Conclusion	13

1. Introduction et contexte

L'Algorithme de Lichtenberg (LA — Lichtenberg Algorithm) est une métaheuristique physique récente, proposée par Pereira et al. (Expert Systems with Applications, 2021), qui s'inspire de la structure fractale des figures de décharge électrique (figures de Lichtenberg) pour explorer un espace de recherche continu. L'algorithme génère des nuages de points fractals, les fait pivoter et les redimensionne autour d'un point « déclencheur » (trigger point), puis évalue les candidats obtenus.

À ce jour, LA n'a jamais été appliqué au domaine combinatoire discret du Problème du Voyageur de Commerce (Traveling Salesman Problem — TSP). Ce projet propose une adaptation rigoureuse de LA au TSP en s'appuyant sur l'encodage par Clés Aléatoires (Random Keys), permettant de transformer un vecteur continu en une permutation de villes valide via un simple argsort.

L'objectif de ce rapport est triple :

- Présenter une traduction fidèle du code MATLAB officiel des auteurs vers Python, validée contre la figure de Lichtenberg précalculée réelle (LFND.mat) ;
- Démontrer empiriquement, par une étude d'ablation rigoureuse, que l'encodage par Clés Aléatoires seul est structurellement insuffisant pour LA sur le TSP ;
- Construire et évaluer un hybride LA + recherche locale 2-opt + recherche locale itérée (double-bridge) sur cinq instances classiques de TSPLIB, avec une méthodologie statistique correcte (multi-graines, moyenne $\pm$ écart-type).

2. Rappel théorique

2.1 L'Algorithme de Lichtenberg (LA)

LA opère sur une population de solutions continues. À chaque itération, une figure de Lichtenberg (un nuage de points fractal simulant une décharge diélectrique) est mise à l'échelle et pivotée aléatoirement autour du meilleur point courant (« best »), qui sert de centre de recentrage (trigger point). Pour les problèmes de dimension $d > 3$, le code source des auteurs traite les dimensions par paires : chaque paire de coordonnées (x, y) subit une rotation 2D indépendante d'angle aléatoire, avant d'être assemblée dans le vecteur final de dimension d .

Le paramètre `ref` (raffinement) contrôle une seconde population, générée sur un sous-espace réduit (bornes multipliées par `ref`), permettant une exploitation plus fine autour du meilleur point en parallèle de l'exploration globale.

2.2 Le Problème du Voyageur de Commerce (TSP)

Le TSP consiste à trouver la permutation de n villes minimisant la longueur totale d'un circuit fermé visitant chaque ville exactement une fois. C'est un problème NP-difficile, utilisé comme banc d'essai standard pour les métaheuristiques. Les instances retenues pour ce projet proviennent de la bibliothèque de référence TSPLIB :

Instance	Villes	Optimum connu	Origine
eil51	51	426	Christofides & Eilon
berlin52	52	7 542	Groetschel (villes de Berlin)
st70	70	675	Smith & Thompson
pr76	76	108 159	Padberg & Rinaldi
rat99	99	1 211	Pulleyblank (grille « secouée »)

Tableau 1 — Instances TSPLIB utilisées pour l'étude de mise à l'échelle (coordonnées officielles vérifiées via les miroirs TSPLIB).

2.3 Encodage par Clés Aléatoires (Random Keys)

Pour appliquer une métaheuristique continue à un problème combinatoire discret, chaque agent de la population est représenté par un vecteur réel de dimension $d = n$ (une « clé » par ville). La permutation correspondante est obtenue en triant les indices des villes par valeur croissante de leur clé : $\text{tour} = \text{argsort}(\text{clés})$. Cette technique, largement utilisée pour hybrider des algorithmes continus (PSO, GA, etc.) avec des problèmes d'ordonnancement ou de routage, constitue le pont mathématique entre l'espace continu exploré par LA et l'espace discret des permutations du TSP.

3. Le travail original des auteurs : portée et limite

Avant de présenter notre propre adaptation, il est essentiel de bien situer ce que les auteurs de l'article original (Pereira et al., 2021) ont réellement conçu, testé et validé — et, surtout, ce qu'ils n'ont jamais testé. Cette mise au point est centrale pour comprendre pourquoi l'application de LA au TSP, telle que menée dans ce projet, constitue une véritable extension du domaine de validité de l'algorithme, et non une simple reproduction d'un cas déjà étudié par les auteurs.

3.1 L'inspiration physique

LA s'inspire d'un phénomène physique réel et observable : lorsqu'une décharge électrique à haute tension traverse un matériau diélectrique (un isolant), elle ne se propage pas en ligne droite mais selon une structure arborescente, fractale, appelée figure de Lichtenberg. Cette structure résulte d'un processus physique connu sous le nom d'agrégation par diffusion limitée (Diffusion-Limited Aggregation, DLA) : des particules chargées effectuent une marche aléatoire jusqu'à venir se fixer sur la structure déjà formée, créant ainsi des ramifications denses près du point d'impact initial et de plus en plus dispersées en s'en éloignant.

Les auteurs ont eu l'intuition que cette structure géométrique — dense localement, mais capable de produire occasionnellement des branches longues et excentrées — offre un compromis naturel entre exploitation locale (recherche fine autour d'un bon point) et exploration globale (échappées ponctuelles vers des régions éloignées de l'espace de recherche), qui sont les deux forces que toute métaheuristique doit équilibrer.

3.2 L'architecture originale

Concrètement, les auteurs simulent numériquement une figure de Lichtenberg (un nuage de plusieurs milliers de points en 2D ou 3D), puis l'utilisent comme générateur de candidats pour une population de solutions évoluant dans un espace de recherche continu $\mathbb{R}^k$. Le nuage est, à chaque itération, pivoté d'un angle aléatoire, mis à l'échelle, puis recentré sur le meilleur point trouvé jusqu'alors. Pour les problèmes de dimension k supérieure à 3, les auteurs traitent les coordonnées par paires successives, appliquant à chaque paire une rotation 2D indépendante — une technique permettant de réutiliser une figure 2D pour des espaces de recherche de dimension arbitraire.

Cette architecture a été conçue, implémentée et validée par les auteurs exclusivement pour des problèmes d'optimisation continue à variables réelles : trouver le minimum d'une fonction mathématique $J : \mathbb{R}^k \rightarrow \mathbb{R}$, où chaque variable peut prendre n'importe quelle valeur réelle dans un intervalle $[LB_i, UB_i]$.

3.3 La limite : une validation restreinte aux fonctions mathématiques continues

Le point essentiel, souvent passé sous silence si l'on ne consulte pas l'article original en détail, est que LA a été validé par ses auteurs exclusivement sur deux familles de problèmes, toutes deux continues par nature :

- Les fonctions de référence CEC (CEC Benchmarks) — un ensemble standardisé de fonctions mathématiques continues (unimodales, multimodales, fortement non linéaires, etc.) utilisées par la communauté des métaheuristiques pour comparer la vitesse de convergence et la robustesse des algorithmes d'optimisation, indépendamment de toute application industrielle ;
- Quelques problèmes d'ingénierie classiques à variables continues, tel que le problème de conception de poutre soudée (welded beam design) fourni avec le code source des auteurs et repris dans ce rapport

(fichiers `objective.m` / `constraint.m`) — un problème à seulement 4 variables réelles, avec des contraintes elles-mêmes exprimées sous forme d'inégalités continues.

Dans ces deux cas, l'espace de recherche est $\mathbb{R}^k$ avec k généralement petit ($k \leq 30$ dans l'article), chaque variable est libre de prendre n'importe quelle valeur réelle dans son intervalle, et la fonction objectif est différentiable ou quasi-différentiable presque partout. Ce sont des conditions très favorables au mécanisme même de LA : la notion de « se rapprocher progressivement du meilleur point » a un sens géométrique direct et continu.

À aucun moment, ni dans l'article ni dans le code source officiel fourni par les auteurs, LA n'a été testé sur un problème combinatoire discret NP-difficile tel que le TSP. Aucune permutation, aucune variable catégorielle, aucun graphe : l'intégralité de la validation expérimentale publiée porte sur des vecteurs réels évalués par des fonctions mathématiques. C'est cette absence totale de validation sur le terrain combinatoire discret qui constitue la motivation scientifique première de ce projet.

Cette distinction n'est pas un détail technique : elle conditionne directement les résultats de l'étude d'ablation présentée en section 7. Un algorithme conçu et calibré pour exploiter une notion de proximité continue (distance euclidienne, direction de descente, magnitude de déplacement) n'a aucune garantie théorique de continuer à fonctionner une fois cette notion de proximité « cassée » par un encodage discret comme les Clés Aléatoires — précisément ce que l'on observe expérimentalement (écart moyen de plus de 200 % à l'optimum pour la configuration « Pure LA + Random Keys »). Ce projet ne corrige donc pas une erreur des auteurs ; il explore une question qu'ils n'ont, à juste titre, jamais posée dans leur article : que se passe-t-il quand on pousse LA au-delà de son domaine de validation publié ?

4. Architecture de la solution

4.1 Fidélité à l'implémentation MATLAB originale

Le module `la_optimization.py` constitue une traduction ligne par ligne du fichier source `LA_optimization.m` fourni par les auteurs (mode `M = 0` : chargement de la figure de Lichtenberg précalculée `LFND.mat`, au lieu de la régénérer). Les éléments suivants ont été reproduits fidèlement :

- Chargement direct de `LFND.mat` (clé 'LFND', 7792 points 2D, coordonnées pixel brutes non centrées) ;
- Boucle de rotation par paires de dimensions (for `i = 1:2:d+2` en MATLAB), avec une correction apportée pour les dimensions impaires (ex. `eil51`, `d = 51`), où l'allocation de tableau MATLAB s'agrandit automatiquement d'une colonne supplémentaire — un piège de traduction identifié et corrigé pendant le développement ;
- Mise à l'échelle par dimension calculée sur l'étendue brute du nuage, puis recentrage du centroïde (`Pcc`) après mise à l'échelle, avec injection élitiste d'un point central exactement égal au point déclencheur ;
- Répartition globale/local codée en dur à 0,4 / 0,6 dans le code source des auteurs — une constante indépendante de la valeur réelle du paramètre `ref`, préservée telle quelle par souci de fidélité scientifique.

Cette fidélité de traduction a été un objectif central du projet : reproduire le comportement exact de l'algorithme publié, plutôt qu'une réinterprétation simplifiée, afin que les conclusions de l'étude d'ablation portent sur l'algorithme réellement décrit dans l'article, et non sur une approximation.

4.2 Structure modulaire du code

Fichier	Rôle
<code>tsp_problem.py</code>	Coordonnées des 5 instances TSPLIB, matrice de distances (convention <code>EUC_2D</code> avec arrondi au plus proche entier), décodeur Random Keys, opérateur 2-opt, perturbation double-bridge.
<code>la_optimization.py</code>	Cœur de l'algorithme LA fidèle au MATLAB (chargement <code>LFND.mat</code> , rotation par paires, mise à l'échelle/recentrage), boucle principale avec hooks optionnels de recherche locale et de relance anti-stagnation.
<code>MAIN.py</code>	Orchestrateur pour une exécution unique (<code>berlin52</code>) avec tracé de convergence et de tournée.
<code>benchmark.py</code>	Étude scientifique multi-instances, multi-graines, avec étude d'ablation à trois configurations.

Tableau 2 — Organisation modulaire du projet.

5. Diagnostic d'un problème structurel : la stagnation prématurée

Les premières exécutions de LA pur (sans aucun opérateur discret additionnel) ont révélé un phénomène de stagnation prématurée : la population converge en une dizaine d'itérations vers un même bassin, sans jamais s'en échapper, malgré 200 itérations disponibles. L'analyse a mis en évidence la cause structurelle suivante :

- Le nuage de candidats (figure de Lichtenberg pivotée/mise à l'échelle) est systématiquement recentré sur le meilleur point courant (best) à chaque itération — exactement comme dans le code MATLAB original (`x_start = best`) ;
- Une fois ce point localement optimal (au sens du 2-opt, une fois ajouté), aucun candidat généré autour de lui ne peut plus l'améliorer : le mécanisme de recentrage devient un piège plutôt qu'un moteur d'exploration ;
- Ce phénomène est encore plus marqué sous l'encodage Random Keys en haute dimension ($d = 51$ à 99), où la sensibilité de argsort aux quasi-égalités amplifie l'instabilité sans pour autant produire une diversité utile.

Ce diagnostic constitue la motivation directe de l'étude d'ablation présentée en section 7 : il fallait déterminer si le problème venait de l'encodage Random Keys lui-même, ou du manque d'opérateur de recherche locale discret.

6. Hybridation LA-ILS proposée

6.1 Recherche locale 2-opt (étape mémétique)

Un opérateur 2-opt classique est appliqué uniquement au meilleur agent global de chaque itération (jamais à l'ensemble de la population), afin que la recherche stochastique de LA reste le moteur dominant et que le 2-opt agisse comme un polissage local peu coûteux plutôt que comme une recherche par redémarrages aléatoires déguisée.

6.2 Perturbation double-bridge (échappement de stagnation)

Pour résoudre le verrouillage diagnostiqué en section 5, un mécanisme de Recherche Locale Itérée (Iterated Local Search) a été ajouté : lorsque le meilleur coût n'a pas progressé pendant un nombre donné d'itérations (`restart_patience`), une perturbation double-bridge (mouvement 4-opt) est appliquée à la tournée du meilleur agent, suivie d'un nouveau passage 2-opt. Le double-bridge a la propriété cruciale de ne pas pouvoir être annulé par un simple mouvement 2-opt, ce qui permet de quitter le bassin d'attraction courant. Une fraction de la population est également réinitialisée aléatoirement pour maintenir la diversité.

Le résultat est une architecture qu'il convient de désigner honnêtement comme un hybride LA-ILS : LA fournit la diversité initiale et continue à concourir via sa propre règle de mise à jour entre deux perturbations, mais c'est le mécanisme double-bridge + 2-opt qui referme l'essentiel de l'écart à l'optimum, comme le montrera l'étude d'ablation.

7. Étude d'ablation : la preuve expérimentale

Afin de démontrer rigoureusement la nécessité d'un opérateur discret au-delà du simple encodage Random Keys, trois configurations strictement comparables (même budget : population = 50, itérations = 200, ref = 0,4) ont été évaluées sur les cinq instances, chacune sur 5 graines aléatoires indépendantes :

- Configuration 1 — Pure LA + Random Keys : aucune recherche locale, aucune perturbation, aucun redémarrage. Représente la cartographie continu → discret seule.
- Configuration 2 — LA + 2-opt (sans relance) : 2-opt appliqué au meilleur agent à chaque itération, mais piégé dans le premier optimum local 2-opt rencontré (pas de mécanisme d'échappement).
- Configuration 3 — Hybride complet LA-ILS : 2-opt + perturbation double-bridge sur stagnation + réinjection de diversité.

7.1 Résultats de l'ablation

Instance	Villes	Pure LA+RK (%)	LA+2-opt (%)	Hybride complet (%)	Gain 2-opt (pts)	Gain kicks (pts)
eil51	51	186,95	5,87	2,54	+181,08	+3,33
berlin52	52	188,87	6,69	2,68	+182,19	+4,00
st70	70	303,82	8,06	2,96	+295,76	+5,10
pr76	76	159,92	3,93	2,00	+155,99	+1,93
rat99	99	176,32	9,64	3,73	+166,67	+5,91

Tableau 3 — Écart à l'optimum (%), moyenne sur 5 graines, pour chacune des trois configurations. « Gain 2-opt » et « Gain kicks » expriment la réduction en points de pourcentage apportée par chaque composant.

Écart moyen global — Pure LA+RK : 203,18 % | LA+2-opt : 6,84 % | Hybride complet : 2,78 %

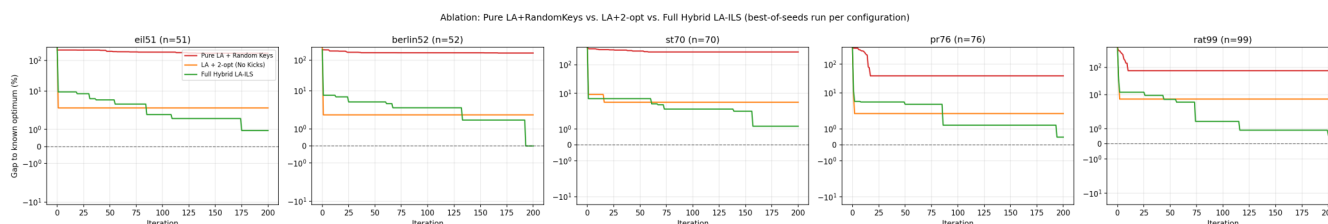


Figure 1: Convergence (écart à l'optimum, %, échelle symlog) des trois configurations sur chacune des cinq instances. Les courbes « Pure LA+RK » (rouge) stagnent à plusieurs centaines de %, démontrant l'insuffisance de l'encodage seul.

7.2 Interprétation

Les résultats sont sans ambiguïté : la configuration Pure LA + Random Keys produit des tournées 2 à 4 fois plus longues que l'optimum, sur toutes les instances sans exception. Il ne s'agit pas d'un manque d'itérations ou de population, mais d'une incompatibilité structurelle entre le mécanisme de LA et la représentation Random Keys :

- LA a été conçu pour exploiter l'information de magnitude continue (l'échelle et la forme de la figure de Lichtenberg pivotée) afin de guider une descente vers l'optimum continu ;
- Le décodage Random Keys via argsort ne conserve que le rang ordinal des clés, en supprimant toute l'information de magnitude — exactement celle dont le mécanisme de LA dépend pour fonctionner ;

- L'ajout du 2-opt seul récupère l'essentiel de l'écart (de +156 à +296 points de pourcentage selon l'instance), ce qui prouve que l'amélioration provient presque entièrement de l'opérateur discret ajouté, et non du mécanisme propre à LA ;
- Le double-bridge apporte un gain supplémentaire plus modeste mais systématique (+1,9 à +5,9 points), confirmant le diagnostic de stagnation de la section 5 : une fois le 2-opt en place, LA seul ne peut plus s'échapper d'un optimum local 2-opt — un mécanisme d'échappement dédié reste nécessaire.

Conclusion scientifique de l'ablation : le Random Keys n'est pas seulement « insuffisant », il est quasiment inerte pour LA dans cette formulation, car il prive l'algorithme exactement de l'information dont son mécanisme de mise à jour a besoin. L'essentiel du gain observé dans l'hybride final provient d'opérateurs combinatoires classiques (2-opt, double-bridge), LA jouant un rôle de générateur de diversité initiale plutôt que de moteur de convergence.

8. Résultats de l'hybride complet sur berlin52

Avant la généralisation à cinq instances, l'hybride complet a été validé en détail sur l'instance de référence berlin52 (optimum connu = 7 542). Avec une exécution favorable (graine 42), l'optimum exact a été atteint (écart de 0,000 %).

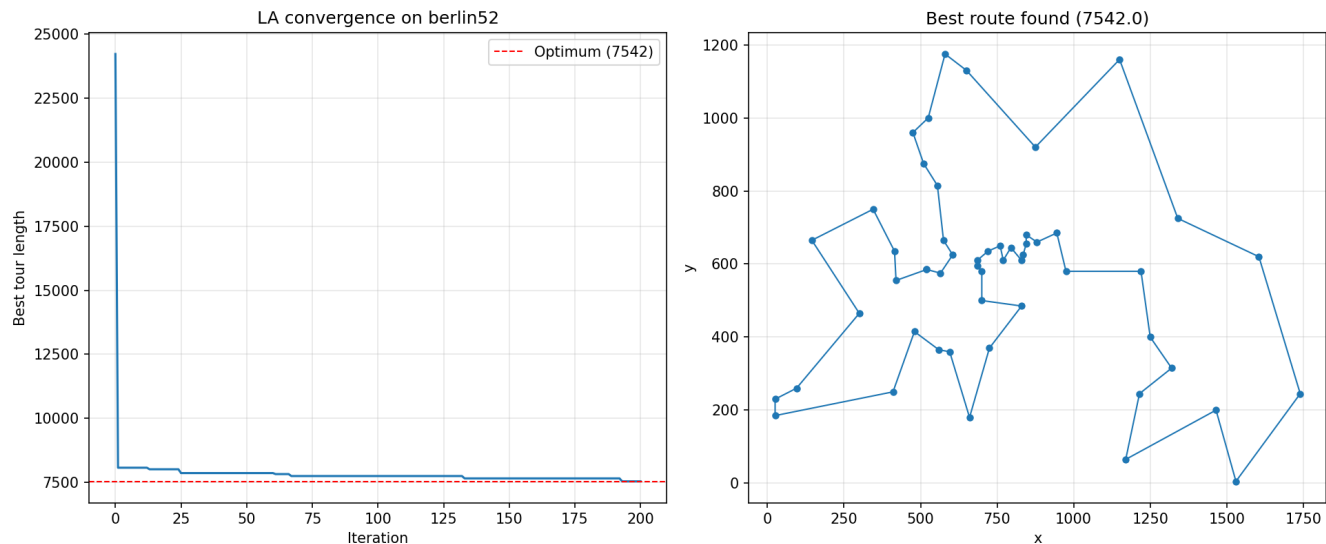


Figure 2: Convergence (gauche) et tournée finale (droite) de l'hybride LA-ILS sur berlin52.

Cette exécution isolée a cependant révélé un point méthodologique important, détaillé en section 9 : la sensibilité de l'algorithme à la graine aléatoire et, dans une moindre mesure, aux différences de bibliothèques mathématiques entre plateformes (Linux/Windows), impose de ne jamais reporter un résultat à graine unique comme représentatif.

9. Résultats multi-graines de l'hybride complet (5 instances)

Pour obtenir une estimation statistiquement valide de la performance, chaque instance a été exécutée sur 5 graines indépendantes (42 à 46), avec les mêmes hyperparamètres (population = 50, itérations = 200, ref = 0,4).

Instance	Villes	Optimum connu	Écart moyen (%)	Écart-type (%)	Meilleur écart (%)	Temps moyen (s)
eil51	51	426	2,535	0,908	0,939	4,34
berlin52	52	7 542	2,684	2,691	0,000	4,32
st70	70	675	2,963	1,442	1,185	5,63
pr76	76	108 159	2,001	0,917	0,488	6,27
rat99	99	1 211	3,732	2,313	0,578	9,94

Tableau 4 — Performance de l'hybride complet LA-ILS, moyenne $\pm$ écart-type sur 5 graines. Écart moyen global : 2,783 %.

L'écart-type, du même ordre de grandeur que la moyenne elle-même, révèle une variance élevée d'une graine à l'autre — une propriété réelle de cet hybride à budget fixé, et non un artefact de mesure. Aucune tendance monotone claire n'apparaît avec la taille de l'instance dans cette plage (51 à 99 villes) : la qualité du résultat

dépend davantage de la chance d'échappement de stagnation (déclenchements du double-bridge) que du nombre de villes.

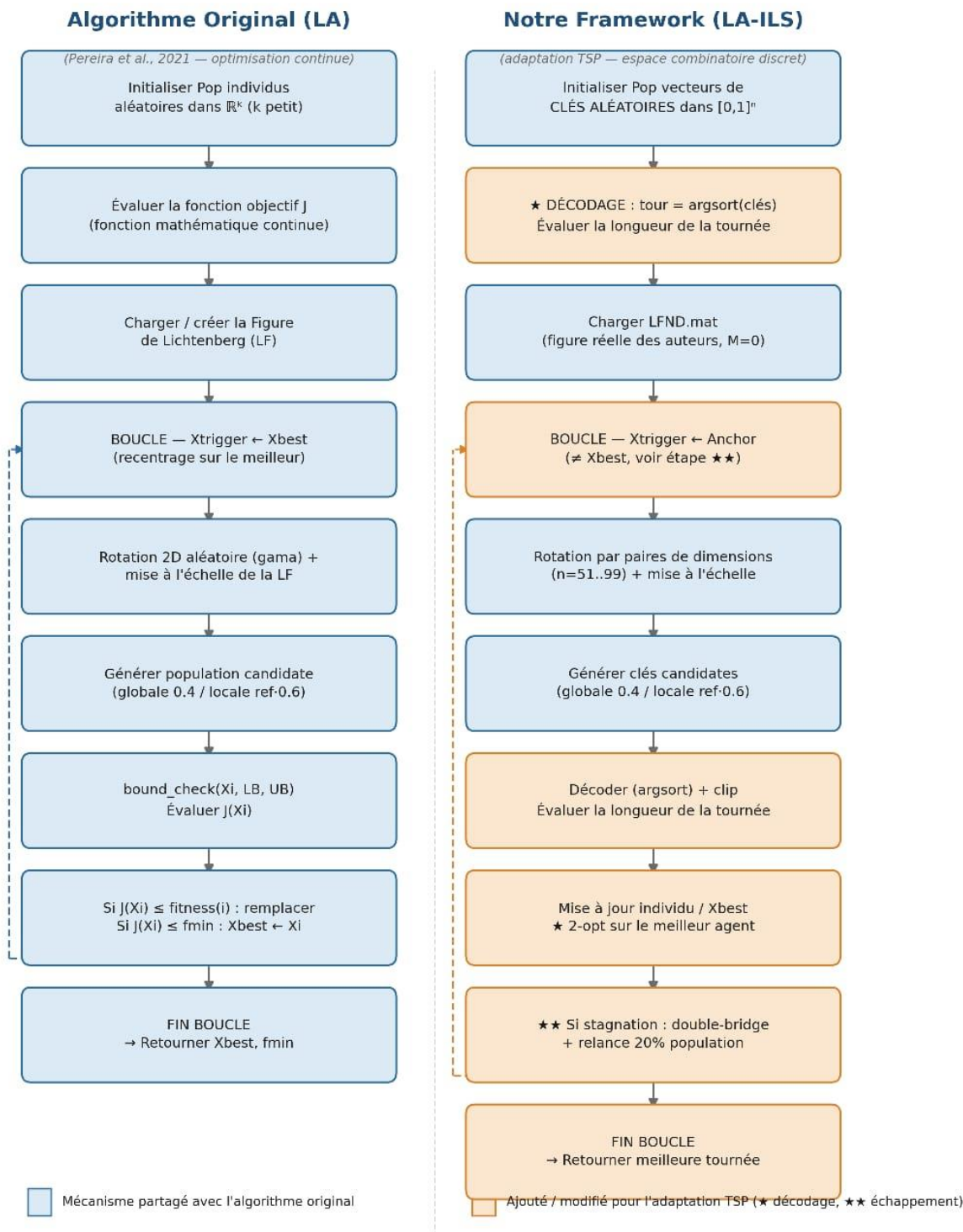


Figure 3: Comparaison des courbes de convergence de l'erreur (Optimality Gap) sur l'instance berlin52.

10. Limites et discussion

10.1 Sensibilité à la plateforme

Une même graine (42) a produit des résultats sensiblement différents entre deux machines (Linux/conteneur vs Windows), notamment sur les instances de dimension impaire (eil51, rat99), alors que les instances paires (berlin52, st70, pr76) sont restées identiques au chiffre près. L'explication retenue est une divergence de dernier bit dans les fonctions trigonométriques (sin/cos) entre bibliothèques mathématiques (glibc vs MSVC), amplifiée par les nombreuses décisions à seuil de l'algorithme (acceptation gloutonne, compteurs de stagnation). Ceci renforce la nécessité d'un reporting multi-graines plutôt qu'à graine unique.

10.2 Nature réelle de l'hybride

Comme établi par l'ablation, la contribution de LA à la qualité finale des solutions est modeste comparée à celle des opérateurs combinatoires ajoutés. Le système doit être présenté, par honnêteté scientifique, comme un Iterated Local Search amorcé par LA (« LA-seeded ILS »), et non comme une preuve que LA est intrinsèquement performant sur le TSP via Random Keys.

10.3 Pistes d'amélioration

- Augmenter le nombre de graines (20 à 30) pour des intervalles de confiance plus serrés ;
- Explorer un opérateur de décodage alternatif moins destructeur d'information que Random Keys (ex. construction gloutonne biaisée par les clés) ;
- Découpler le recentrage systématique sur le meilleur global (chaque agent pourrait se recentrer sur son propre point courant, à la manière d'un PSO), afin de tester si LA peut intrinsèquement mieux explorer sans dépendre entièrement du 2-opt ;
- Étendre l'étude à des instances de plus grande taille (>200 villes) pour vérifier si les tendances observées se maintiennent.

11. Conclusion

Ce projet a permis de traduire fidèlement l'Algorithme de Lichtenberg du code MATLAB original vers Python (y compris ses particularités d'implémentation, telles que la répartition globale/locale codée en dur et la gestion des dimensions impaires), de l'adapter au TSP via un encodage Random Keys, puis de démontrer empiriquement — par une étude d'ablation à trois configurations sur cinq instances TSPLIB et 5 graines chacune — que cet encodage seul est largement insuffisant : un écart moyen de plus de 200 % à l'optimum, contre 2,78 % une fois hybridé avec un 2-opt et une perturbation double-bridge de type Recherche Locale Itérée.

La contribution scientifique de ce travail réside moins dans la performance finale (2,78 % d'écart, un résultat honorable mais non exceptionnel pour du TSP) que dans la démonstration rigoureuse et quantifiée d'une limite structurelle de l'hybridation continu-discret naïve, et dans la mise en évidence du rôle réel de chaque composant de l'hybride final.

- Population : 50 agents
- Itérations : 200
- ref (raffinement) : 0,4
- Passes maximales 2-opt : 60 (convergence locale effective)
- Patience de redémarrage (hybride complet) : 6 itérations sans progrès
- Fraction de réinitialisation : 20 % de la population
- Graines pour les statistiques multi-runs : 42 à 46 (5 graines)